

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA0615	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	V.LASYA	Reg. No.:	192524186
Section / Batch:	B.tech AI &DS	Date:	07.08.2026

Instructions to Students

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Vandana Lasya
192524186RA

Languages: Python C C++ Java

QUESTIONS

Q1
Brute Force
50 marks

Q2
Brute Force
50 marks

Q1 Brute Force 50 marks

Debugging Insertion Sort (Incorrect Index Placement)
The insertion sort code incorrectly places elements due to wrong index update. Identify the logical error in shifting logic. Explain why elements are misplaced. Debug step-by-step. Optimize code readability. Analyze time complexity and rewrite corrected version.
while j >= 0 and arr[j] > key:
arr[j] = arr[j-1] # ERROR
j -= 1

Your Code

```
1 def insertion_sort(arr):  
2     for i in range(1, len(arr)):  
3         key = arr[i]  
4         j = i - 1  
5         while j >= 0 and arr[j] > key:  
6             arr[j+1] = arr[j]  
7             j -= 1  
8         arr[j+1] = key  
9     return arr  
10 n = int(input())  
11 arr = list(map(int, input().split()))  
12 insertion_sort(arr)  
13 print(*arr)
```

Input
5
5 2 4 6 1

Output
1 2 4 5 6

Submitted



SIMATS Online Programming Lab
DAA PROGRAMMING



Languages: Python C C++ Java

QUESTIONS

Q1
Brute Force
50 marks

Q2
Brute Force
50 marks

2/2 answered

Q2 Brute Force 50 marks

Debugging Insertion Sort Code (Wrong Direction in Shift Comparison)

The following Insertion Sort implementation uses an incorrect comparison condition in the while loop, causing smaller elements to remain in the wrong position and breaking ascending order. Carefully analyze the shifting logic and explain why the sorted prefix is not maintained correctly. Debug the algorithm step-by-step so that larger elements shift right and the key is inserted in the proper place. Optimize the implementation for clarity and maintain correct indexing throughout. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean code with proper comments.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] < key: # ERROR
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr
```

Your Code

```
1 def insertion_sort(arr):
2     for i in range(1, len(arr)):
3         key=arr[i]
4         j=i-1
5         while j>=0 and arr[j]>key:
6             arr[j+1]=arr[j]
7             j-=1
8         arr[j+1]=key
9     return arr
10 n=int(input())
11 arr=list(map(int,input().split()))
12 insertion_sort(arr)
13 print(*arr)
```

Input
5
9 5 1 4 3

Output
1 3 4 5 9

Submitted

Code of Conduct

I V.Lasya(192524186) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

V.lasya

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of	Shows reasonable analysis with	Limited analysis; weak reasoning	No analytical reasoning demonstrated

		solutions	some justification		
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____